

Assignment 3: End-to-End ML Versioning with Git, DVC & Google Drive

Author: Abdullah Kamal | Dataset: Fashion-MNIST | Model: Sequential Artificial Neural Network (ANN)

1. Part A — Git Fundamentals & Advanced Commands

A production-grade Git repository was initialized with an incremental branching model (main, dev, hotfix, teammate-sim).

A3. Git Log Variants Analysis:

- `git log --oneline --graph --all`: Renders the entire DAG branching topology, merge history, and branch pointer positions across all branches.
- `git log --stat -3`: Displays file-level change statistics (+/- line modifications) for the 3 most recent commits.
- `git log -p -1`: Generates line-by-line unified diff patches of additions and deletions for the latest commit.
- `git log main..dev`: Isolates commits residing on the feature branch dev that have not yet been integrated into main.

[Terminal] A3. Git Log Graph Across All Branches

```
$ git log --oneline --graph --all
* 835b025 feat(pipeline): add dvc.yaml stage definitions
* b2966ca feat(eval): add evaluate.py for test metrics and confusion matrix
* 9671830 feat(model): add train.py for Sequential ANN architecture
* 135e8bd feat(preprocess): add preprocess.py for normalization and validation split
* b9871c2 feat(data): add prepare.py to download and extract Fashion-MNIST
* 4b5d2c9 feat(config): add params.yaml for centralized hyperparameters
* clb2329 feat: initial commit with README and gitignore
```

A4. Git Diff Variants Analysis:

- `git diff`: Reveals unstaged modifications in working directory compared to index.
- `git diff --staged`: Compares staged files in index against current HEAD commit.
- **Two-Dot vs Three-Dot Diff**: `git diff main..dev` performs a direct end-state comparison between the tips of main and dev. In contrast, `git diff main...dev` compares the tip of dev against the common merge-base ancestor of main and dev, showing strictly what was added on dev.

[Terminal] A6. Rebase dev onto main & Linear Graph

```
$ git rebase main; git log --oneline --graph --all
Rebasing (1/6)
Rebasing (2/6)
Rebasing (3/6)
Rebasing (4/6)
Rebasing (5/6)
Rebasing (6/6)
Successfully rebased and updated refs/heads/dev
+ 4de9d4a feat(pipeline): add dvc.yaml stage definitions
+ 91505dc feat(model): add train.py for Sequential ANN architecture
+ 7447583 feat(data): add prepare.py to download and extract Fashion-MNIST
+ 3c9758a feat(preprocess): add preprocess.py for normalization and validation split
+ c49687d Merge branch 'dev' into main
+ eedc38b fix(docs): improve README header phrasing
+ clb2329 feat: initial commit with README and gitignore
```

A5, A6, A7, A8 Advanced Scenarios:

- **Stash (A5)**: Working state preserved with `git stash`, switched branches, and popped cleanly with `git stash pop`.
- **Rebase (A6)**: Rebased dev cleanly on top of main following a hotfix merge, maintaining a clean linear history.
- **Reset (A7)**: Verified `git reset --soft HEAD~1` leaves uncommitted changes staged in the index, whereas `git reset --hard HEAD~1` completely discards both staged and unstaged working modifications.
- **Reorganize (A8)**: Performed file move via `git mv` and file removal via `git rm`, recording each as an explicit tracked commit event.

2. Part B — Modular TensorFlow ANN Pipeline Architecture

The machine learning workflow is architected into 4 modular standalone scripts orchestrated by `params.yaml` and `dvc.yaml`:

- **src/prepare.py**: Fetches Fashion-MNIST (60k train / 10k test) and saves raw arrays to `data/raw/`.
- **src/preprocess.py**: Normalizes pixel data to `[0, 1]` and executes stratified validation split (80/20 train/val) based on `params.yaml`.
- **src/train.py**: Constructs a fully-connected Sequential ANN (Flatten -> Dense(ReLU) -> Dropout -> Dense(10, Softmax)) compiled with Adam and Sparse Categorical Crossentropy, exporting `models/model.h5` and `models/history.csv`.
- **src/evaluate.py**: Evaluates test set performance, generates `confusion_matrix.png`, and writes `metrics.json`.

`params.yaml` (Hyperparameters Single Source of Truth)

```
preprocess: test_size: 0.2 seed: 42 train:
dense_units: 128 dropout_rate: 0.2
learning_rate: 0.001 epochs: 10 batch_size:
64
```

`dvc.yaml` (Reproducible Pipeline Graph)

```
stages: prepare: cmd: python src/prepare.py deps:
[src/prepare.py] outs: [data/raw] preprocess: cmd: python
src/preprocess.py deps: [data/raw, src/preprocess.py]
params: [preprocess.seed, preprocess.test_size] outs:
[data/processed] train: cmd: python src/train.py deps:
[data/processed, src/train.py] params: [train.batch_size,
train.dense_units, train.dropout_rate, train.epochs,
train.learning_rate] outs: [models/history.csv,
models/model.h5] evaluate: cmd: python src/evaluate.py deps:
[data/processed, models/model.h5, src/evaluate.py] metrics:
[{metrics.json: {cache: false}}] plots:
[{confusion_matrix.png: {cache: false}}]
```

3. Part C — DVC Setup with Google Drive Remote

DVC was initialized with Google Drive remote storage: `gdrive://1ALEBo2C0DXhzrc8mMcp5UK9XUAkd33WT`. Large arrays (`data/raw/`, `data/processed/`) and models (`models/model.h5`) are tracked in DVC cache and pushed to Google Drive, keeping Git repository lightweight.

[Terminal] C3. DVC Remote Configuration

```
$ dvc remote list
gdrive_storage  gdrive://1ALEBo2C0DXhzrc8mMcp5UK9XUAkd33WT  (default)
```

4. Part D — DVC Pipeline Execution & Hyperparameter Tuning

D3. Baseline Reproduction Log (v1 Pipeline - dense_units: 128):

```
[Terminal] D3. DVC Repro (v1 Baseline Pipeline Execution)

$ dvc repro
Running stage 'prepare':
+ python v1/train.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218011 62725 5056 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Python\Python311\site-packages\requests\_init_.py:111: RequestDependencyWarning: urllib3 (2.5.8) or chardet (7.4.3)/charset-normalizer (3.4.2) doesn't match a supported version!
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218040 64398 5056 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
Running stage 'preprocess':
+ python v1/cvdp/preprocess.py
Running stage 'train':
+ python v1/train.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218057 60029 24064 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Python\Python311\site-packages\requests\_init_.py:111: RequestDependencyWarning: urllib3 (2.5.8) or chardet (7.4.3)/charset-normalizer (3.4.2) doesn't match a supported version!
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218060 60033 24064 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Python\Python311\site-packages\keras\src\layers\tf_keras\tracing\tracing.py:27: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
super().__init__(**kwargs)
10000 80.00 1791218078 53000 24064 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
... [truncated for display] ...
```

D4. Hyperparameter Modification & Selective Re-execution (dense_units: 256):

When `train.dense_units` was increased to 256 in `params.yaml`, running `dvc repro` demonstrated DVC's DAG caching intelligence: **prepare** and **preprocess** stages were skipped (cache hit) because their dependencies, code, and hyperparameters did not change. Only **train** and **evaluate** were re-executed because their parameter hash and input model dependencies changed.

```
[Terminal] D4. DVC Repro with Hyperparameter Update (dense_units: 256)

$ dvc repro
Stage 'prepare' didn't change, skipping
Stage 'preprocess' didn't change, skipping
+ python v1/cvdp/preprocess.py
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218059 64148 14660 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Python\Python311\site-packages\requests\_init_.py:111: RequestDependencyWarning: urllib3 (2.5.8) or chardet (7.4.3)/charset-normalizer (3.4.2) doesn't match a supported version!
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
10000 80.00 1791218060 70138 14660 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
C:\Users\LENOVO\AppData\Local\Microsoft\Windows\Python\Python311\site-packages\keras\src\layers\tf_keras\tracing\tracing.py:27: UserWarning: Do not pass an 'input_shape' / 'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the first layer in the model instead.
super().__init__(**kwargs)
10000 80.00 1791218075 64440 14660 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:tensorflow:GPU support is not available on native Windows for TensorFlow = 2.11. Even if CUDA/cudnn are installed, GPU will not be used. Please use WSL2 or the TensorFlow-DirectML plugin.
Epoch 1/10
In 17/7500m [37m=====]0m [133:32]0m 36/step - accuracy: 0.9156 - loss: 2.5527
In 18/7500m [37m=====]0m [144:0m 6m/step - accuracy: 0.9201 - loss: 1.9744
In 19/7500m [37m=====]0m [146:0m 6m/step - accuracy: 0.9388 - loss: 1.8688
In 20/7500m [37m=====]0m [146:0m 6m/step - accuracy: 0.9551 - loss: 1.7283
In 34/7500m [37m=====]0m [146:0m 6m/step - accuracy: 0.9284 - loss: 1.6140
In 41/7500m [37m=====]0m [146:0m 6m/step - accuracy: 0.9440 - loss: 1.5365
... [truncated for display] ...
```

D5. Metrics Comparison Table (v1 vs v2):

Experiment Version	Dense Units	Dropout	Learning Rate	Epochs	Test Accuracy	Test Loss
Version 1 (v1 baseline)	128	0.2	0.001	10	87.37%	0.3450
Version 2 (v2 tuned)	256	0.2	0.001	10	87.49%	0.3461

5. Part E — Simulated Collaboration & Conflict Resolution

A multi-contributor workflow was simulated with concurrent changes to preprocessing code and generated data across branches `main` and `teammate-sim`.

E1 & E2. Concurrent Divergence: `teammate-sim` modified normalization to z-score standardization while `main` applied `[-1, 1]` scaling.

E3. Conflict Diagnosis: Merging `teammate-sim` produced a code conflict in `src/preprocess.py` and a hash conflict in `dvc.lock`.

[Terminal] E3. Git Merge Conflict in Code & dvc.lock

```
$ git merge teammate-sim && git status
Auto-merging dvc.lock
CONFLICT (content): Merge conflict in dvc.lock
Auto-merging src/preprocess.py
CONFLICT (content): Merge conflict in src/preprocess.py
Automatic merge failed; fix conflicts and then commit the result.
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)
Unmerged paths:
  (use "git add <file>..." to mark resolution)
   both modified:   dvc.lock
   both modified:   src/preprocess.py
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  Assignment3 Report.pdf
no changes added to commit (use "git add" and/or "git commit -a")
```

E4 & E5. Resolution & Pipeline Verification:

- Code conflict was resolved by standardizing on canonical `[0, 1]` scaling.
- Data synchronization was performed by executing `dvc repro` to generate deterministic hashes for the merged state.
- Merge commit completed, and `dvc` status verified clean data synchronization.

[Terminal] E4-E5. Conflict Resolved & Clean DVC Repro

```
$ git commit -m "Merge branch 'teammate-sim'..." && dvc status
[main 216a208] Merge branch 'teammate-sim': resolved code conflict in preprocess.py and synchronized dvc data
$ dvc status
Data and pipelines are up to date.
```

Final Git Commit Graph (Unsquashed History):

[Terminal] Final Git Repository Tree

```
$ git log --oneline --graph --all
* 216a208 Merge branch 'teammate-sim': resolved code conflict in preprocess.py and synchronized dvc data
|
+ 2f09b6f feat(preprocess): teammate standardization normalization
|
+ 3614e1e feat(preprocess): main branch min-max scaling to [-1, 1]
|
+ 93792fc Merge dev branch (v2 pipeline) into main
|
+ a4da01d experiment: tune dense units to 256 (v2)
+ 5facc71 feat: record dvc pipeline v1 reproduction and lockfile
+ 791eadd chore(dvc): initialize dvc and configure Google Drive remote
+ 81260e9 refactor: remove obsolete helper using git rm
+ 013000f refactor: move helper to src2 using git mv
+ 9dc1d19 chore: add temporary scratch helper
+ ad95d4e feat(pipeline): add dvc.yaml stage definitions
+ 017731e feat(eval): add evaluate.py for test metrics and confusion matrix
+ b356d0c feat(model): add train.py for sequential ANN architecture
+ 728d657 feat(preprocess): add preprocess.py for normalization and validation split
+ 9247053 feat(data): add prepare.py to download and extract Fashion-MNIST
+ 3c9c234 feat(config): add params.yaml for centralized hyperparameters
|
+ c49687d Merge branch hotfix into main
... [truncated for display] ...
```